



SLOTLI · DESIGN DOCUMENT

Simple appointment booking

An account-based appointment-booking manager for service providers — bookings, clients and invoices in one place, plus an embeddable booking widget whose submissions appear automatically.

VERSION	1.0
TYPE	Appointment-booking app
TECH	HTML · CSS · JS · Firebase · EmailJS
INTERFACE	Bilingual (HU/EN)

Table of contents

01	Overview
02	Goals & audience
03	Tabs & features
04	The booking widget
05	Workflow
06	Architecture & tech
07	Data model
08	Storage & security
09	Design system
10	File structure
11	Setup
12	Future work

Overview

Slotli is an account-based appointment-booking manager for service providers. It does two jobs: an admin panel tracks incoming bookings live, and it generates an embeddable booking widget the owner can drop onto their own website.

Data is stored per account in the cloud (Firebase). No build step, no framework: plain HTML, CSS and JavaScript. The interface is bilingual (Hungarian/English) and uses the shared "Studio" design with a blue-teal accent, in light/dark.

Philosophy: no server of your own. Static hosting + Firebase (Auth + Firestore) + EmailJS — maintenance-free operation, bookings sync live.

Goals & audience

Audience

Providers who work by appointment (hairstylist, beautician, trainer, etc.) who want to take bookings on their own website.

Goal

Clear, collision-free booking management in one place, with revenue tracking — without separate calendar software.

Visitors can only pick genuinely free slots; a booking reserves the time immediately, with collision protection.

Tabs & features

TAB	FUNCTION
Overview	Today's bookings, upcoming slots, a last-12-months revenue chart.
Bookings	Incoming bookings live, with statuses; a cancellation frees the slot.
Clients	An automatic client base from bookings, with contact details.
Invoices	Issuing invoices and printable PDF from the booked services.
Booker	Configuring the widget: services, opening hours, slot length, fields.
Account	Business data, tax, appearance, language, the embed code and OWNER_UID.

The booking widget

On the setup tab you define each service with its duration and price, the daily opening hours (with several windows, e.g. a lunch break) and the slot length.

The widget computes the offered times from these, and visitors can only pick genuinely free slots. Paste the code once; the config comes from the Firestore `form_configs/<uid>` document, and a booking is written under `inbox/<uid>/items`.

A booking reserves the slot immediately, with collision protection: if two people try the same time at once, the second one fails.

Workflow

Booking

The visitor picks a free slot in the embedded widget; the booking is recorded immediately.

Live pull

The booking appears live in the account (onSnapshot), with a status; a cancellation frees the slot again.

Confirmation

Automatic e-mail to the client and the provider via EmailJS; an invoice can be made from it if needed.

All e-mails are sent from the user's browser, with no separate mail server.

Architecture & tech

Client

Vanilla HTML/CSS/JS, no build step. Auth + app core, booking setup, widget generator, bilingual, light/dark theme.

Cloud

Firebase Authentication + Cloud Firestore (with live listening). Static hosting for the site; e-mails via EmailJS.

Script load order: Firebase SDK → firebase-store.js → script.js (+ bookings.js, invoices.js, theme.js, fit-text.js).

Data model

Each user's full state lives in a single Firestore document (`vaults/<uid>`); the embedded widget uses two extra collections.

```
// vaults/<uid> - main parts of the state
{
  services: [ ... ], hours: { ... }, bookings: [ ... ],
  clients: [ ... ], invoices: [ ... ],
  profile, settings, ...
}

form_configs/<uid>    // published booker config
inbox/<uid>/items     // incoming bookings
```

The data layer (`firebase-store.js`) makes the Firestore calls; it loads on sign-in and listens to bookings live (`onSnapshot`).

Storage & security

Private data: `vaults/<uid>` is read/write only for the signed-in owner.

Controlled public surface: `form_configs` is readable (for the widget), while `inbox` is write-only from outside — bookings can only be read by the owner.

Public keys: the Firebase keys are browser-public identifiers; no secret key reaches the client.

The Firestore rules live in the `firebase/` folder and deploy via the CLI.

Design system

A modern, clean "Studio" design — a teal primary with a blue accent (the calendar emblem's palette), rounded cards, light/dark theme.

COLOUR PALETTE

	Teal (primary)	#0e9aac
	Deep teal	#0b7f8f
	Blue (accent)	#1670c4
	Light base	#e8f2f6
	Ink	#0f1e26
	Line	#cfe2ea

Heading Nunito · 500–800	Body Inter · 400–700	Mono IBM Plex Mono
------------------------------------	--------------------------------	------------------------------

Components: teal accent on buttons, blue secondary highlights, rounded cards; large figures are sized by fit-text.js.

File structure

```
03_Slotli/  
├─ index.html  
├─ css/style.css  
├─ js/ (script, bookings, invoices, theme, firebase-store, fit-text)  
├─ img/ (logo, favicon)  
├─ email-sablonok/ (EmailJS templates)  
├─ firebase/ (config + rules)  
└─ dokumentumok/ (design doc, setup)
```

Setup

- Firebase: project, Authentication (email/password), Firestore; fill the config in index.html and deploy the rules from the firebase/ folder.
- EmailJS: configure the service + templates (see BEALLITAS-EmailJS.txt).
- Publish: upload the files to static hosting (GitHub Pages).
- Embed: paste the code generated on the Booker tab into your own website.

Future work

- Calendar sync (Google/Outlook) and reminder e-mails.
- Online deposit/booking payment.
- Multiple staff and resources.

Thanks to the modular, framework-free structure, extensions can be added gradually without disrupting operation.